

Section C — Aggregation Documentation

This section focuses on summarizing and analyzing data using SQL aggregate functions.

The main objective of this section is to practice:

- Counting records
- Calculating total values
- Finding averages
- Finding maximum and minimum values
- Grouping data using GROUP BY
- Filtering grouped results using HAVING

Aggregation is important for converting raw data into meaningful business insights.

Database Selection

Query

```
USE shopease_db;
```

Explanation

This command selects the ShopEase database before running aggregation queries.

It ensures all operations are executed on the correct database.

Q13. Count total number of orders

Query

```
SELECT COUNT(*) AS total_orders  
FROM orders;
```

Explanation

This query counts the total number of records in the orders table.

COUNT(*) counts every row.

The alias total_orders makes the result easier to read.

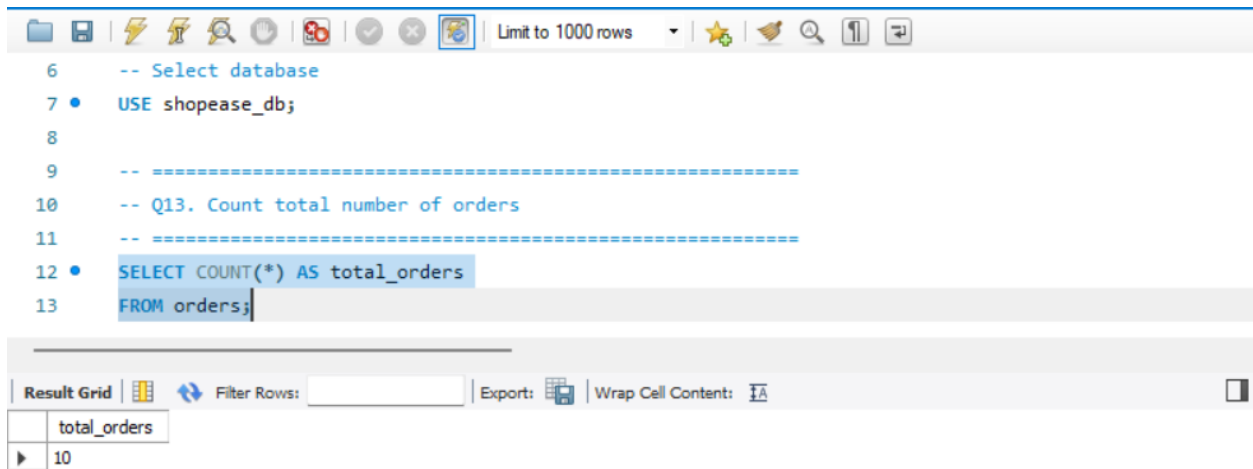
Purpose

Helps determine how many orders have been placed.

Expected Output

A single value showing the total order count.

Screenshot



Q14. Total revenue from all Delivered orders

Query

```
SELECT SUM(total_amount) AS total_delivered_revenue
FROM orders
WHERE status = 'Delivered';
```

Explanation

This query calculates the total revenue from orders that have been successfully delivered.

SUM(total_amount) adds all matching order values.

The WHERE clause filters only delivered orders.

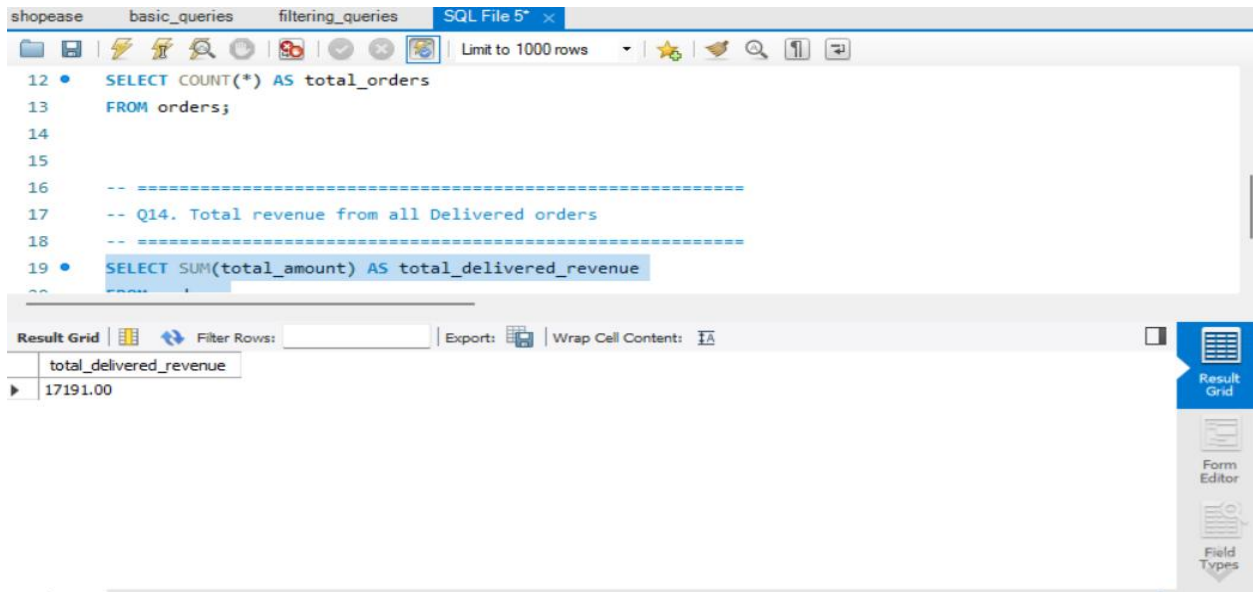
Purpose

Helps measure actual completed sales revenue.

Expected Output

A single total revenue amount.

Screenshot



Q15. Average product price in each category

Query

```
SELECT
    category,
    AVG(unit_price) AS average_price
FROM products
GROUP BY category;
```

Explanation

This query calculates the average price for each product category.

AVG(unit_price) finds the average.

GROUP BY category groups products by their category before calculating the average.

Purpose

Helps compare pricing across categories.

Expected Output

Shows average product prices for:

- Electronics
- Clothing
- Home

Screenshot

The screenshot shows a SQL query editor with a query window and a result grid. The query is as follows:

```
23
24  -- =====
25  -- Q15. Average product price in each category
26  -- =====
27  • SELECT
28      category,
29      AVG(unit_price) AS average_price
30  FROM products
```

The result grid displays the following data:

category	average_price
Clothing	2699.000000
Electronics	1879.200000
Home	949.000000

The interface includes a toolbar at the top with various icons, a 'Limit to 1000 rows' dropdown, and a 'Read Only' status indicator at the bottom right.

Q16. Count orders and total revenue by order status

Query

```
SELECT
    status,
    COUNT(order_id) AS total_orders,
    SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

Explanation

This query groups orders based on their status.

For each status, it calculates:

- total number of orders
- total revenue

ORDER BY total_revenue DESC sorts results from highest revenue to lowest.

Purpose

Helps analyze which order statuses generate the most revenue.

Expected Output

Shows:

- Delivered
- Shipped
- Pending
- Cancelled

with their order counts and revenues.

Screenshot

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
31 GROUP BY category;
32
33 -- =====
34 -- Q16. For each order status:
35 -- Find count of orders and total revenue
36 -- Sort by total revenue (highest first)
37 -- =====
38 SELECT
```

The result grid displays the following data:

status	total_orders	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	1	1299.00

The interface also includes a toolbar with various icons, a 'Limit to 1000 rows' dropdown, and a 'Read Only' status indicator at the bottom right.

Q17. Find highest and lowest product price in each category

Query

SELECT

```
category,  
MAX(unit_price) AS highest_price,  
MIN(unit_price) AS lowest_price
```

```
FROM products
```

```
GROUP BY category;
```

Explanation

This query finds:

- highest product price using MAX()
- lowest product price using MIN()

for each product category.

GROUP BY category ensures calculations are done category-wise.

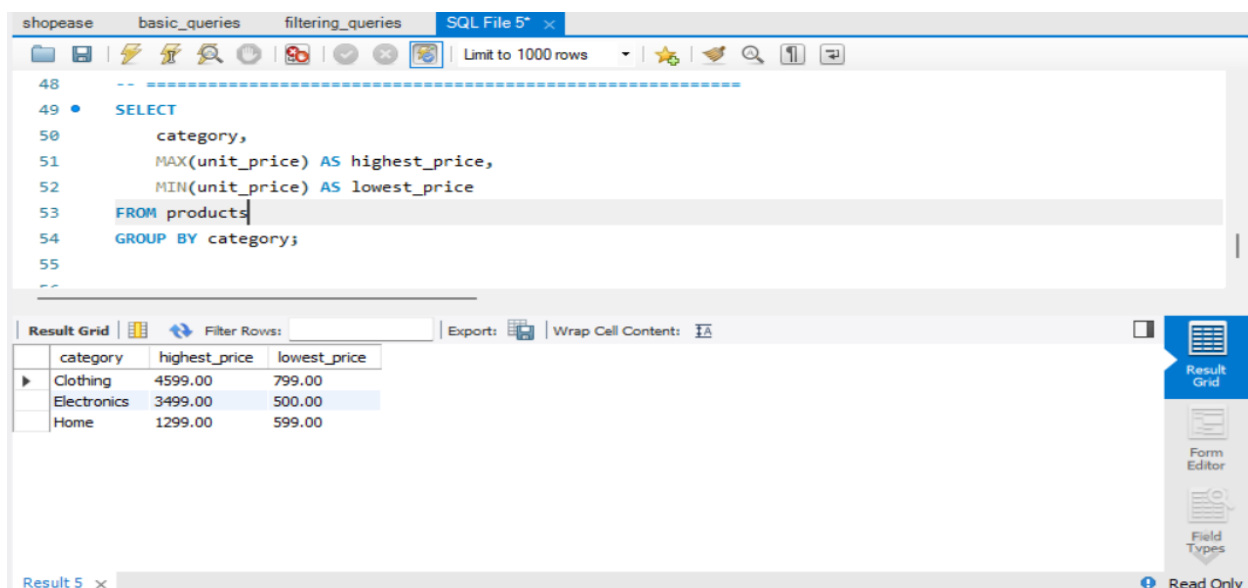
Purpose

Helps understand the pricing range in each category.

Expected Output

Displays minimum and maximum prices for each category.

Screenshot



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
--  
49 • SELECT  
50     category,  
51     MAX(unit_price) AS highest_price,  
52     MIN(unit_price) AS lowest_price  
53 FROM products  
54 GROUP BY category;  
55  
--
```

The result grid displays the following data:

category	highest_price	lowest_price
Clothing	4599.00	799.00
Electronics	3499.00	500.00
Home	1299.00	599.00

The IDE interface includes a toolbar with various icons, a "Limit to 1000 rows" dropdown, and a "Result Grid" button. The status bar at the bottom indicates "Result 5" and "Read Only".

Q18. Categories where average price is greater than ₹2000

Query

```
SELECT
    category,
    AVG(unit_price) AS average_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;
```

Explanation

This query calculates average product price for each category.

The HAVING clause filters grouped results.

Difference:

- WHERE filters rows before grouping
- HAVING filters groups after aggregation

Only categories with average price greater than ₹2000 are shown.

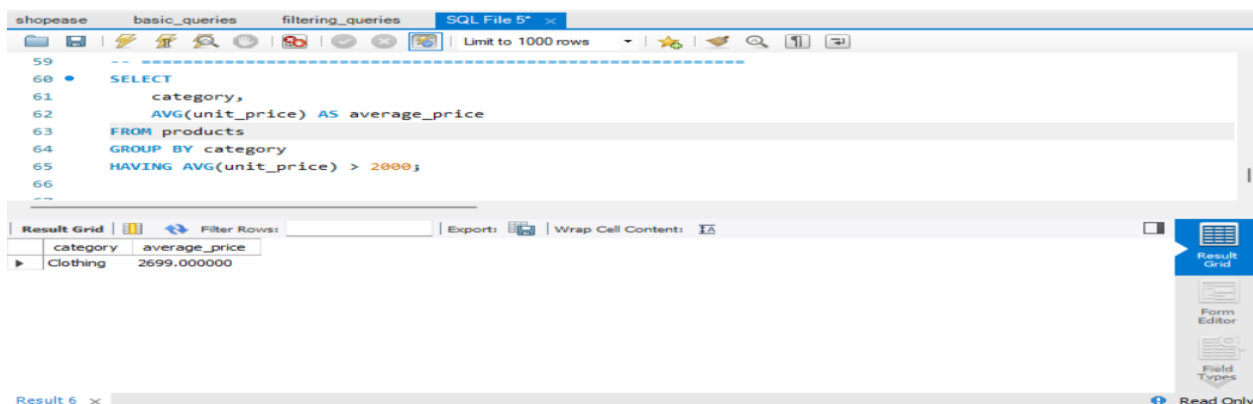
Purpose

Helps identify high-value product categories.

Expected Output

Displays categories with average price above ₹2000.

Screenshot



Key Concepts Covered

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()
- GROUP BY
- HAVING
- ORDER BY
- Aggregate functions

Summary

In this section, aggregate functions were used to summarize business data.

The queries helped analyze:

- total orders
- revenue
- average pricing
- product price ranges
- order status performance

These operations are essential for business reporting and data analysis.